

SOFTWARE ENGINEERING LAB-3

Name:Sairam Charan N

SRN:PES1UG24AM240

Branch/Class: AiML/5E

Components:

1. Commuter Interface
2. Fleet Controller Interface
3. Tracking / Telemetry Engine
4. ETA Calculation Service
5. Crowding Management Service
6. Bus / Route Data Store

Interfaces:

#	Interface	Connection	Purpose
I1	GPS / Telemetry Data	Bus → Tracking/Telemetry Engine	Sends GPS coordinates
I2	Location Data	Tracking Engine → ETA Service	Supplies latest location for ETA
I3	Ticketing / Sensor Data	Sensor/Ticketing → Crowding Service	Supplies data for crowd calculation
I4	ETA Information	ETA Service → Commuter Interface	Displays ETA
I5	Crowding Status	Crowding Service → Commuter Interface	Displays crowding
I6	Bus Monitoring Data	Tracking/Crowding Services → Fleet Controller Interface	Allows controller to monitor buses

SOFTWARE ENGINEERING LAB-3

Architecture Analysis

Factor	Layered Architecture	Microservices Architecture	Client–Server Architecture
<i>How it would work for our system</i>	We could separate the commuter/controller interface, bus-tracking logic and data into different layers.	We could have separate services for tracking, ETA, crowding and other functions.	The commuter and Fleet Controller would communicate with one central server that handles the system's main functions.
<i>What works well</i>	It is simple to understand and keeps different responsibilities separated.	Tracking, ETA and crowding can be handled separately, and each service can be scaled when needed.	It is simple to deploy and keeps the system's data and control centralized.
<i>Problem for our case</i>	Our system continuously receives data from buses, so passing everything through layers may add processing overhead. It is also harder to scale only the tracking part.	It is more complicated to manage because several services have to communicate with each other.	With up to 1,000 buses, a central server could become a bottleneck. If it fails, many parts of the system could be affected.
<i>1,000 buses</i>	Moderate — possible, but scaling individual functions is difficult.	Strong — the tracking service can be scaled independently when telemetry increases.	Moderate — the central server may become overloaded.
<i>5-second requirement</i>	Moderate — processing through multiple layers may add overhead.	Strong — separate services can process tracking and ETA independently, although communication between services must be managed.	Moderate — a heavily loaded central server could affect response time.
<i>Overall for Case 24</i>	Good, but not the best fit	Best fit	Good, but has scalability concerns

Final Choice For Architecture: Microservices Architecture

SOFTWARE ENGINEERING LAB-3

Justification of the Choice:

- We selected Microservices Architecture because our Bus Tracking System has several different functions such as GPS tracking, ETA calculation, crowding calculation, commuter access, and Fleet Controller monitoring.
- It supports a large number of buses. Our system has to handle telemetry from up to 1,000 active buses at the same time. With microservices, the tracking service can be scaled separately if the number of buses or GPS updates increases.
- Each major function can have its own service. For example, GPS tracking, ETA calculation, and crowding calculation can be separated instead of putting all the processing into one large application.
- It helps with system maintenance. If we need to change the ETA calculation, we can work on the ETA service without having to modify the entire system.
- It provides better fault isolation. If one service, such as the crowding service, has a problem, the other services such as GPS tracking or ETA calculation can continue working. The lab identifies fault isolation as an advantage of microservices.
- It helps us meet the 5-second response requirement. Our system needs to provide the latest bus location, ETA, and crowding information within 5 seconds. Separating the major processing tasks allows the high-demand services to be handled independently.
- It gives flexibility for future growth. If the system later needs more buses or additional features, individual services can be expanded without redesigning the whole application.
- There are some drawbacks, such as greater operational complexity, network communication between services, and possible data-consistency issues. However, for our Case 24, the benefits of independent scaling and separation of responsibilities make Microservices the most suitable choice.

SOFTWARE ENGINEERING LAB-3

Component and Interface Diagram

